

Testes automatizados com Serenity BDD

Alunos: Mateus Almeida e Matheus Lobo

Apresentando a estrutura do Serenity BDD

Serenity BDD (Behavior-Driven Development) é uma biblioteca de código aberto que ajuda a escrever testes de aceitação automatizados de alta qualidade. Atua como um "wrapper" (invólucro) para ferramentas de teste populares, enriquecendo suas capacidades com relatórios poderosos e descrições de teste claras e narrativas.

- **Breve Histórico:** Evoluído do framework Thucydides, o Serenity BDD tem sido continuamente desenvolvido para atender às demandas dos testes de software modernos.
- **Posicionamento na Pirâmide de Testes:** Embora seja capaz de suportar testes unitários e de API, o Serenity BDD realmente se destaca no domínio dos testes de UI e integração, fornecendo insights abrangentes sobre as funcionalidades voltadas para o usuário.



Principais Recursos do Serenity BDD



Relatórios Automatizados e Visuais

Gera relatórios detalhados, em estilo narrativo, que são fáceis de entender tanto para stakeholders técnicos quanto não-técnicos, mostrando os resultados dos testes e a cobertura dos requisitos.



Reusabilidade de Passos

Promove a criação de "Definições de Passo" reutilizáveis, reduzindo a duplicação de código e tornando os testes mais fáceis de manter e ler em diversos cenários.



Suporte a Page Object Model

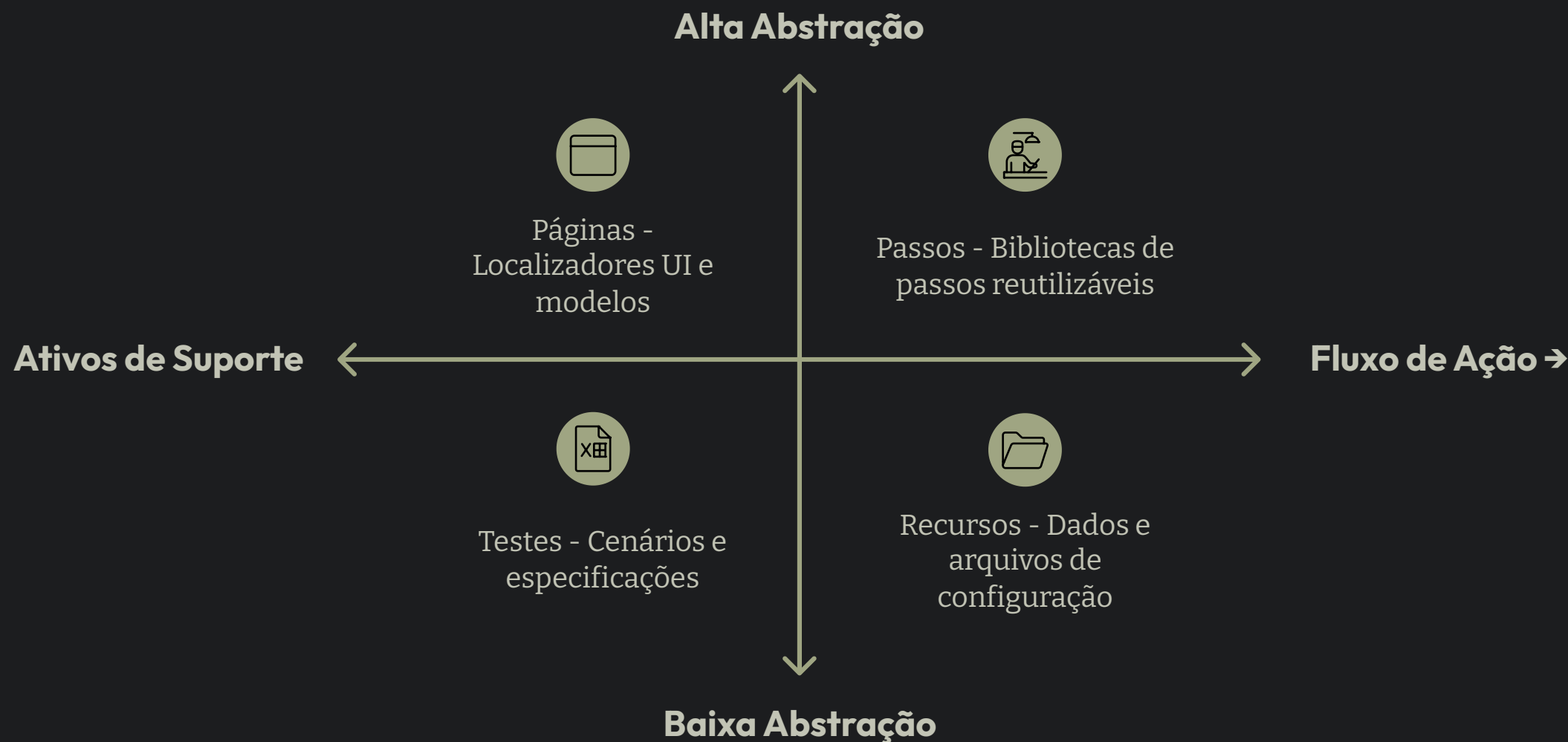
Forte suporte ao Page Object Model, um padrão de design que torna os testes de UI mais robustos e fáceis de gerenciar, abstraindo os elementos e interações da página.



Integração Contínua

Integra-se facilmente com frameworks e ferramentas de teste populares como JUnit, Cucumber e Selenium WebDriver, aproveitando suas forças enquanto adiciona sua própria capacidade de relatórios.

Arquitetura e Estrutura do Projeto

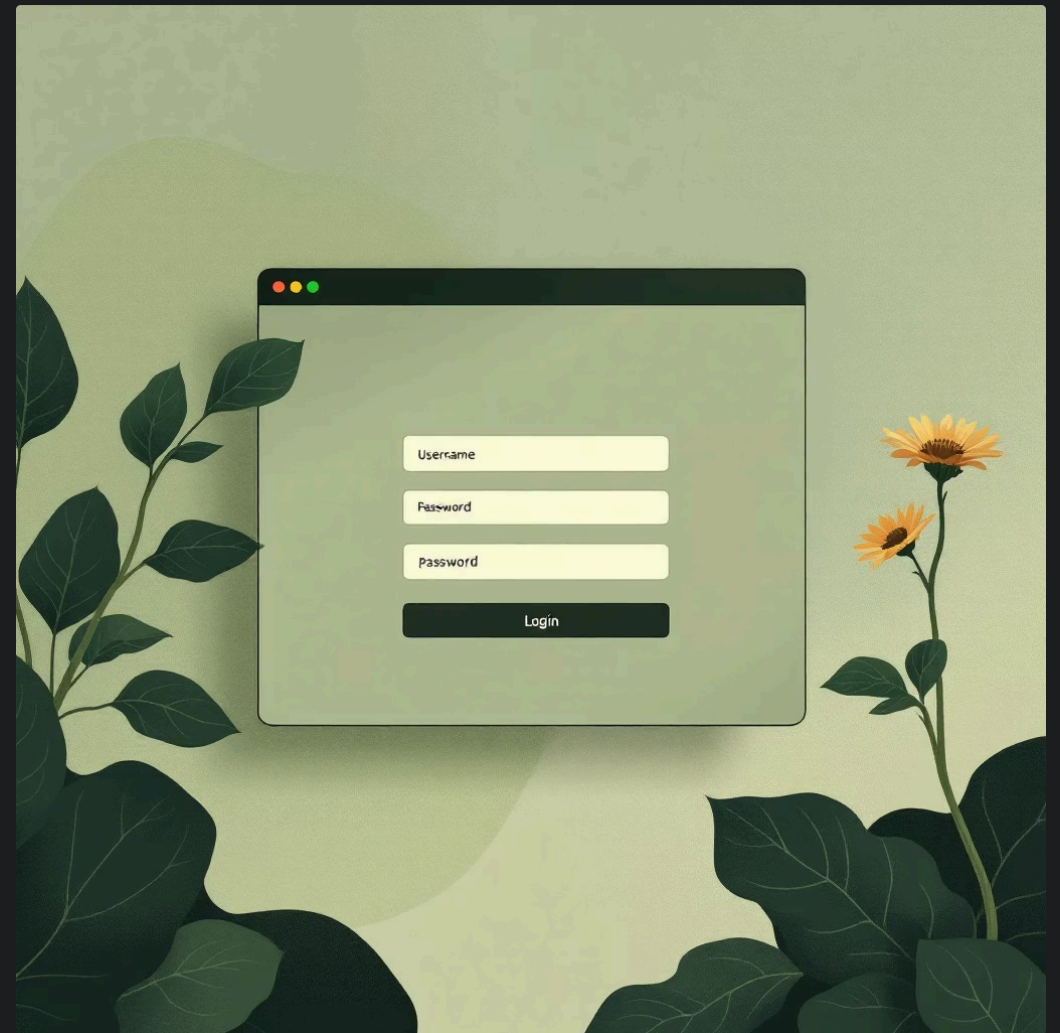


Essa estrutura organizada oferece benefícios significativos, incluindo melhor legibilidade, manutenção mais fácil e melhor colaboração entre os membros da equipe.

Exemplo de Código: Page Object

O Modelo de Objeto de Página ajuda a encapsular elementos e interações da interface do usuário. Aqui está um exemplo simplificado de como `LoginPage.java` pode mapear elementos em uma tela de login.

```
public class LoginPage extends PageObject {  
    @FindBy(id="username")  
    WebElementFacade usernameField;  
  
    @FindBy(id="password")  
    WebElementFacade passwordField;  
  
    @FindBy(css=".login-button")  
    WebElementFacade loginButton;  
  
    public void enterUsername(String username) {  
        usernameField.type(username);  
    }  
  
    public void enterPassword(String password) {  
        passwordField.type(password);  
    }  
  
    public void clickLogin() {  
        loginButton.click();  
    }  
}
```



Esta abordagem isola as alterações da interface do usuário, o que significa que se o localizador de um elemento mudar, apenas o Page Object precisa ser atualizado, e não todos os testes que usam esse elemento.

Exemplo de Código: Passos

A classe `LoginSteps.java` contém métodos que representam ações do usuário, agindo como uma camada de abstração sobre os Objetos de Página.

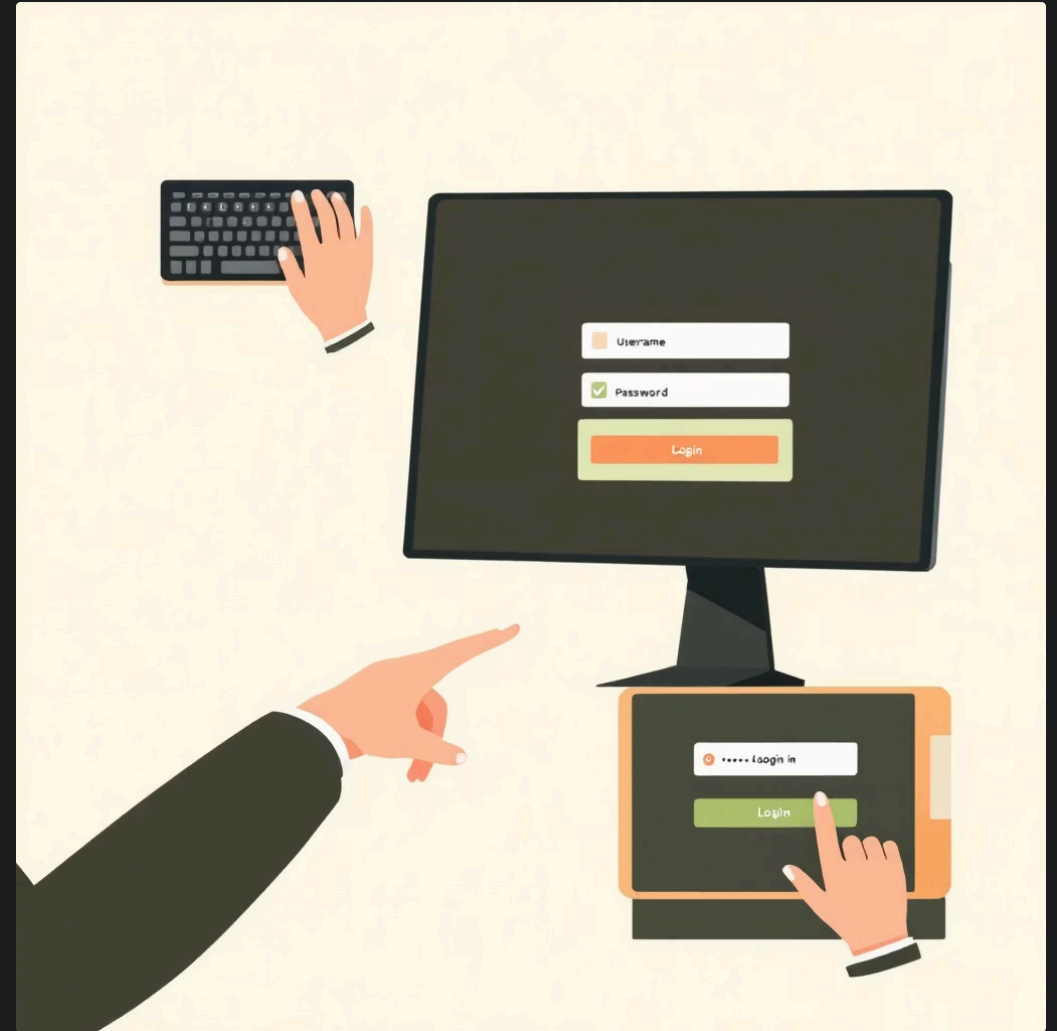
```
public class LoginSteps {
    LoginPage loginPage;

    @Step("Enters username: {0}")
    public void entersUsername(String username) {
        loginPage.enterUsername(username);
    }

    @Step("Enters password: {0}")
    public void entersPassword(String password) {
        loginPage.enterPassword(password);
    }

    @Step("Clicks login button")
    public void clicksLoginButton() {
        loginPage.clickLogin();
    }

    @Step("Logs in with username {0} and password {1}")
    public void logsIn(String username, String password) {
        entersUsername(username);
        entersPassword(password);
        clicksLoginButton();
    }
}
```



Cada método anotado com `@Step` se torna uma entrada legível no relatório do Serenity BDD, fornecendo uma narrativa clara da execução do teste.

Exemplo de Código: Teste

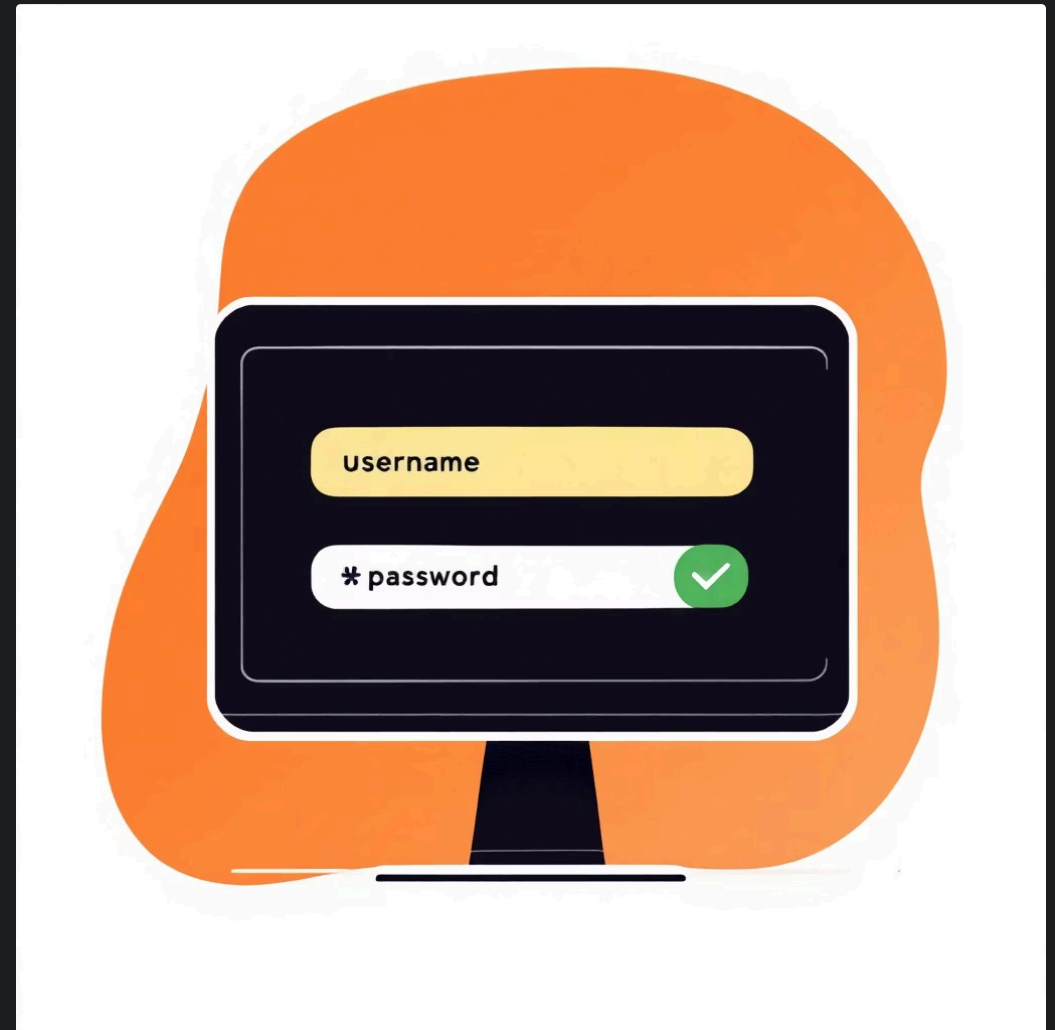
Finalmente, o `LoginTest.java` orquestra o cenário de teste chamando os métodos de passo.

```
@RunWith(SerenityRunner.class)
public class LoginTest {

    @Managed(driver = "chrome")
    WebDriver driver;

    @Steps
    LoginSteps user;

    @Test
    public void userCanLoginSuccessfully() {
        driver.get("http://your-app-url.com/login");
        user.logIn("validUser", "password123");
        // Assertions for successful login go here
    }
}
```



Este teste descreve claramente o cenário: navegar para a página de login, executar as ações de login e, em seguida, validar o resultado, tudo através de passos concisos e legíveis.



Execução e Relatórios

Executar testes Serenity BDD é simples, geralmente gerenciado via Maven.

- **Como Executar:** Basta rodar `mvn clean verify` a partir da raiz do seu projeto. O Maven compilará seus testes, os executará e, em seguida, gerará os relatórios do Serenity.
- **Local do Relatório:** Após a conclusão da execução, navegue até `target/site/serenity/index.html` no diretório do seu projeto para visualizar os relatórios abrangentes.

Exemplo de um Relatório Serenity Gerado: Esses relatórios incluem tempos de execução, capturas de tela, detalhamento passo a passo e até vídeos da execução do teste, oferecendo uma visibilidade incomparável do seu conjunto de testes.

```
raan lamal;  
{  
  mvn clean verify  
  
  mvn clean verify (wittle mix  
  
<laureal green)  
with latark "milthonl verrif)  
  toxth,  
}  
laurel green, warm dark
```


Serenity BDD: Vantagens e Considerações

Relatórios Claros e Ricos

Fornece relatórios altamente detalhados e narrativos que preenchem a lacuna de comunicação entre equipes técnicas e não-técnicas.

Organização Aprimorada de Testes

Incentiva boas práticas como o Page Object Model e a reutilização de passos, levando a suítes de teste mais sustentáveis e escaláveis.

Integrações Perfeitas

Funciona bem com ferramentas existentes e populares como Selenium, Cucumber e JUnit, aproveitando seus pontos fortes.

Curva de Aprendizagem

A adoção do Serenity BDD pode envolver uma curva de aprendizagem inicial, especialmente para equipes não familiarizadas com os princípios do BDD ou a estrutura específica do framework.

Dependência de Java/Maven

Construído principalmente para projetos Java e se integra profundamente com o Maven, o que pode ser uma restrição para ambientes que não utilizam Java.



Conclusão e Recomendações

Serenity BDD é uma ferramenta inestimável para equipes que visam alcançar testes automatizados de alta qualidade, legíveis e de fácil manutenção. Seus relatórios abrangentes e abordagem estruturada promovem melhor colaboração e uma compreensão mais clara do comportamento da aplicação.

- **Quando Adotar:** Ideal para projetos que exigem relatórios robustos, BDD colaborativo e automação complexa de UI, especialmente dentro do ecossistema Java.
- **Casos de Sucesso:** Muitas empresas em todo o mundo utilizam o Serenity BDD para aprimorar a qualidade de seus softwares e acelerar seus pipelines de entrega.

Obrigado pela atenção. Há alguma pergunta?